

信息抽取实验报告

规则系统与 GLiNER 联合抽取技能结构化信息

acknowledge 项目实验记录

2026 年 5 月 29 日

摘要

这次实验做的是一个偏工程型的信息抽取系统：从 `skills.sh` 的技能描述里抽出平台、技术栈、动作类型、目标领域、输出格式和量化指标六类信息。系统没有直接押宝在一个模型上，而是拆成三层：baseline 做精确关键字匹配，enhanced-regex 在此基础上补 action alias、受控词表归一化和 fallback，enhanced-gliner 再把 GLiNER 作为前置候选生成器接进来。

按当前工作区里能复现的结果，`ground_truth.json` 上的整体 F1 从 baseline 的 37.66% 提到 enhanced-regex 的 60.36%；当前 enhanced 输出对应的 GLiNER 版本 F1 为 59.78%。也就是说，这批标注样本里真正把分数拉起来的还是规则增强层，GLiNER 没有继续抬高总分，反而在 `output_formats` 上带来一点精度损失。不过把范围放回全量 1000 条数据，GLiNER 依然留下了 1624 条 `gliner_direct` 和 582 条 `gliner_normalized` 证据，说明它并不是完全没用，而是卡在词表和阈值这一层。

1 引言

这个子系统要解决的问题可以写得很具体：给一条技能描述，自动整理出它服务哪个平台、用什么技术、做什么动作、面向什么场景、最后产出什么格式。检索系统拿到这样的结构化结果后，界面展示和后续统计都会轻松很多，后面如果接问答或推荐，也不用每次都从长文本里硬抠关键词。

数据还是同一批 1000 条 `skills.sh` 记录。字段除了 `skill_name`、`description`、`category` 这些主键，还包含外置的 `SKILL.md` 文本路径。当前快照里 917 条能读到外置技能文档，83 条缺 `SKILL.md`，203 条缺简介。文本语言整体偏英文，标题和描述同时含 CJK 的记录非常少，所以抽取系统里专门加了一个 `english_only_bias` 开关，避免 GLiNER 在非英文文本上乱猜。

这次实验最关心三件事：第一，baseline 和 enhanced-regex 到底各修掉了哪些问题；第二，GLiNER 接进来以后，收益是不是落在评测集能测到的地方；第三，如果模型命中了 span 但最后还是没被系统接住，瓶颈究竟出在词表、阈值还是后处理。

2 数据获取与预处理

2.1 抓取与 checkpoint 流程

IE 系统用的数据不是另外造的，而是直接吃 `paqu.py` 写下来的共享数据集。抓取流程和 IR 报告一致，这里仍然给出一张图，方便后面解释为什么有的文本来自 `description`，有的来自外置 `SKILL.md`。

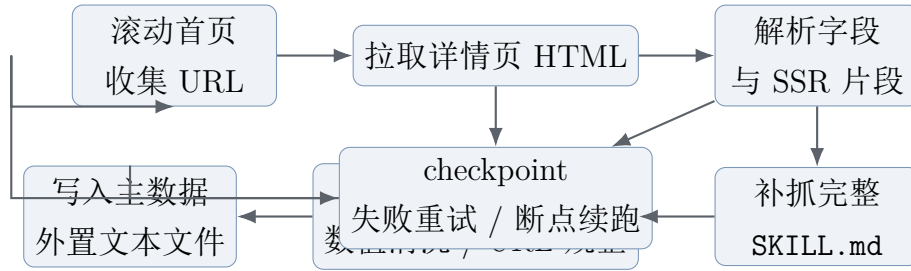


图 1: IE 使用的数据准备链路：抓取和清洗在共享爬虫阶段完成，抽取模块拿到的是已经规范化过的主字段和外置 `SKILL.md` 文本。

2.2 字段与文本来源

抽取阶段真正关心的是表 1 这些字段。它们分别用于构造输入文本、辅助规则匹配和记录结果来源。

表 1: IE 子系统直接使用的主要字段

字段名	作用	非空条数
<code>skill_name</code>	事件标题，构造 <code>full_text</code> 与 <code>gliner_text</code>	1000
<code>description</code>	短摘要，GLiNER 的主要上下文之一	797
<code>category</code>	类别先验，常和领域词一起进入输入文本	1000
<code>skill_md_text_path</code>	长文本主来源，覆盖率最高	917
<code>skill_md_raw_text_path</code>	保留原始换行，便于正则抽取 metrics	917
<code>detail_url</code>	回看样本和误差分析时的定位键	1000

代码里读文本的优先级是

```
external_skill_text > skill_md_raw_text > skill_md > description.
```

这一步很关键。像 `maishou` 这种技能，如果只看简介，平台名单根本不完整；而像 `domain-authority-audio` 这种长规则说明文档，如果直接把整段都丢给 `baseline`，又会触发一堆过宽的动词和领域词。

2.3 预处理：文本包与语言偏置

抽取前，系统会为每个文档构造三个视图：

- `full_text`: `skill_name + category + extraction_text`，供完整规则匹配使用；
- `gliner_text`: 压缩过的聚焦文本，避免超长输入把 GLiNER 拖慢；
- `fallback_text`: GLiNER 失败或跳过后回退使用的文本。

语言偏置判断写得很直白。设文本里的 ASCII 字母数为 n_{ascii} ，CJK 字符数为 n_{cjk} ，则

$$\text{english_dominant}(x) = \begin{cases} 1, & n_{\text{cjk}} = 0, \\ 1, & n_{\text{ascii}} \geq 1.5 n_{\text{cjk}}, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

当 `english_only_bias=true` 且式 (1) 判成 0 时，GLiNER 直接跳过，系统退回规则路径。这个判断不花哨，但和当前数据的语言分布是对得上的。

3 核心算法

3.1 三阶段结构

这套 IE 系统不是单一模型，而是三层递进关系。图 2 画出了它们的区别。

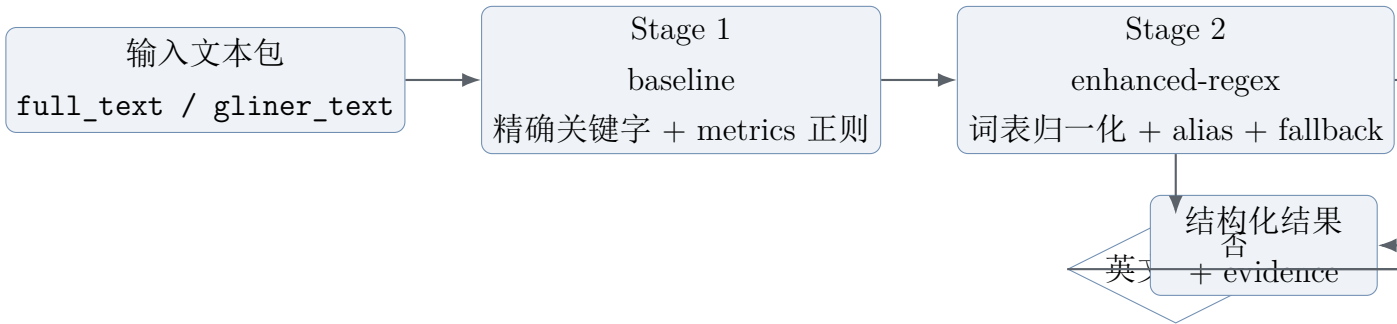


图 2: IE 三阶段结构：baseline 先给出纯规则底线，enhanced-regex 负责把 alias、归一化和 fallback 接起来，GLiNER 则只在英文主导文本上提供额外候选。

实际跑下来，第二层提升最大，第三层并没有稳定继续加分。这一点后面看结果时会非常明显。

3.2 baseline：精确关键字与 metrics 正则

baseline 对五个枚举字段都走 `\bkeyword\b` 形式的正则精确匹配，配置项来自 `ie_config.json`。这样做的好处是行为很可审计，坏处也同样明显：词表一旦偏窄，FN 会很多；词表一旦偏宽，FP 也会很多。

metrics 字段单独处理，不靠关键字表，而是靠四组模式去抓“数字 + 单位”：

表 2: metrics 字段使用的主要正则模式与示例

模式	典型命中示例
<code>(\d+)[-\\s]*(signal criteria item signal, 12-criteria</code>	
<code>(?:across over with covering spanning dim(ensi)ons+...</code>	
<code>(\d+)\s*[\---]\s*(\d+)\s*(?:scaled smaller range rating)</code>	
<code>supports five modules</code> 这类英文数字模式	<code>supports five modules</code>

这部分设计得挺实用，因为 metrics 往往混在说明文字里，硬列词表意义不大。

3.3 enhanced-regex: alias 扩展与受控词表归一化

第二层增强主要做两件事。第一件是 action alias 扩展，例如 analysis、analytics、analyzer 都归到 analyze，writer、writing、drafting 都归到 create。第二件是受控词表归一化，顺序是：

$$\text{exact} \rightarrow \text{folded} \rightarrow \text{alias} \rightarrow \text{fuzzy} \rightarrow \text{unknown}.$$
 (2)

folded 匹配会把大小写和符号折叠，例如 node.js 和 nodejs 归到同一项；alias 匹配再处理 github.com → github、jd → jd.com 这种别名；最后还有一个编辑距离不超过 2 的模糊匹配，用来救一下明显的轻微拼写变体。

这层增强是这次实验里真正的主力。拿 maishou 来说，baseline 只能抓到 1688，enhanced-regex 则能把 taobao、tmall、jd.com、pinduoduo、vip.com 一次性补齐。

3.4 GLiNER 集成与阈值门控

第三层把 GLiNER 接进来做候选实体提取 [Zaratiana et al.(2023)Zaratiana, Tomeh, Holat, and Cha它在系统里的位置不是“直接产出最终答案”，而是“先提 span，再让规则层决定要不要收下”。整条链路见图 3。

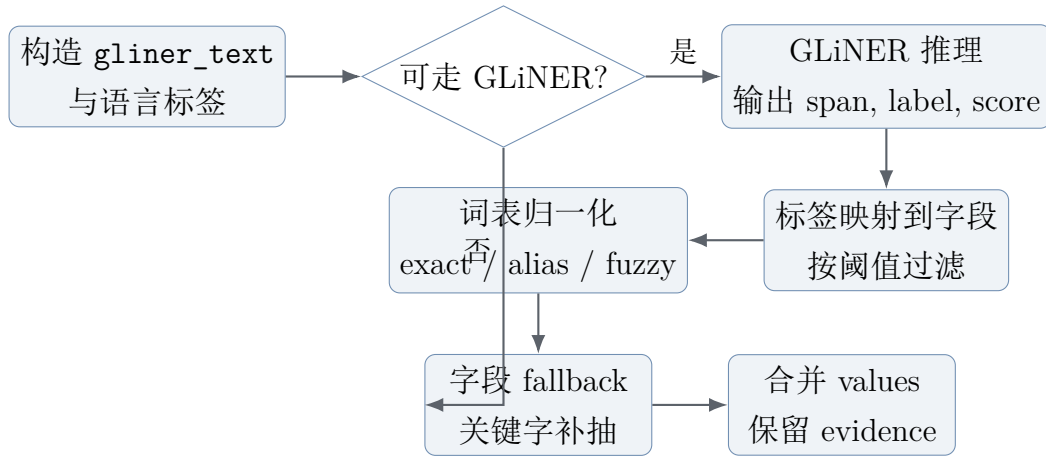


图 3: GLiNER 在系统里的实际位置：模型只负责给候选 span 和分数，真正决定保留什么字段的仍然是阈值、受控词表和 fallback 规则。

字段阈值是手工设的，表 3 给出当前配置。

表 3: GLiNER 各字段的置信度阈值

字段	阈值
platforms	0.4
languages	0.3
action_types	0.5
target_domains	0.5
output_formats	0.3
metrics	0.6

形式上可以写成

$$\text{accept}(x, f) = \begin{cases} 1, & \text{score}(x, f) \geq \tau_f, \\ 0, & \text{score}(x, f) < \tau_f. \end{cases} \quad (3)$$

如果一个 span 过了阈值，但归一化后找不到合法词表项，系统不会静默吞掉，而是把它记成 `gliner_unknown`。这一点对误差分析很有用。

3.5 增强版抽取流程伪代码

算法 1 把第三层增强流程压缩成伪代码。它和真实实现最接近的地方，是把“模型命中”和“规则回退”放在了同一条流水线上，而不是两个独立系统。

算法 1: enhanced 抽取主流程

输入: 文档 d **输出:** 抽取结果 E 与证据映射 V

```

1  $B \leftarrow \text{BUILDTEXTBUNDLE}(d)$ 
2 if  $\text{variant} = \text{baseline}$  then
3   return  $\text{EXACTKEYWORDEXTRACT}(B.\text{full\_text})$ 
4 if  $B.\text{gliner\_eligible}$  且模型可用 then
5    $P \leftarrow \text{PREDICTGLINER}(B.\text{gliner\_text})$ 
6    $H \leftarrow \text{PROJECTBYFIELD}(P, \tau_f)$ 
7 else
8    $H \leftarrow \emptyset$ 
9 foreach 枚举字段  $f$  do
10  if  $H[f]$  有可归一化命中 then
11     $E[f] \leftarrow \text{NORMALIZE_TO\_VOCAB}(H[f])$ 
12     $E[f] \leftarrow \text{MERGE\_FALLBACK}(E[f], B.\text{fallback\_text})$ 
13  else
14     $E[f] \leftarrow \text{FALLBACK\_EXTRACT}(f, B.\text{fallback\_text})$ 
15  $E[\text{metrics}] \leftarrow \text{METRIC\_REGEX}(B.\text{full\_text}, H[\text{metrics}])$ 
16 return  $(E, V)$ 

```

4 实验设计与过程

4.1 实验环境

实验仍然在 Windows 10 (10.0.26100) 上完成, conda 环境是 yolov82, Python 版本 3.11.5。GLiNER 按配置跑在 CPU 上,模型缓存目录位于 `ie_system/runtime/gliner_cache/`。第一次加载模型时冷启动很明显,在本机上频繁全量重跑并不轻松,这也是我后面尽量复用现有输出文件的原因。

4.2 标注集与评测办法

自动评测集是 `ie_system/eval/ground_truth.json`, 共 10 条 skill, 每条标注五个枚举字段: `platforms`、`languages`、`action_types`、`target_domains`、`output_formats`。虽然系统内部也抽 `metrics`, 但它的值是 `{value, unit}` 字典列表, 直接做集合交并不合适, 所以自动评测把它排除在外。

Precision、Recall、F1 的定义仍然是集合式写法：

$$P = \frac{|Y \cap \hat{Y}|}{|\hat{Y}|}, \quad (4)$$

$$R = \frac{|Y \cap \hat{Y}|}{|Y|}, \quad (5)$$

$$F1 = \frac{2PR}{P + R}, \quad (6)$$

其中 Y 是人工标注集合， $\hat{Y}$ 是系统抽取集合。

这里有个实际问题必须记下来：当前数据快照里，标注集中的 `pdf-converter` 不在 `skills_data.json` 里，所以自动评测最终只匹配到 9 条文档。这个坑不是算法问题，是数据版本没完全对齐。为了把三阶段表做完整，我保留了现有 `runtime` 结果，同时补跑了不启用 GLiNER 的 `regex` 阶段来对齐数值。

4.3 运行命令

Listing 1: IE 子系统的主要运行命令

```
python ie_system/skills_ie_system.py extract --variant enhanced
python ie_system/skills_ie_system.py evaluate --variant enhanced --eval-set
    ground_truth.json
python ie_system/skills_ie_system.py compare --eval-set ground_truth.json
python ie_system/skills_ie_system.py report --variant enhanced
```

其中：

- `comparison_report.json` 更适合看 `baseline` 和 `regex` 增强层；
- `evaluation_report.json` 与 `output/extraction_results.json` 对应当前 `enhanced` 输出；
- `extraction_report.json` 记录的是全量 1000 条文档的覆盖率和证据来源。

5 实验结果与对比分析

5.1 三阶段总表

表 4 给出三阶段的总结果。这里的 Stage 2 是我按当前代码补跑的 `enhanced-regex`；Stage 3 使用的是当前仓库里已经落盘的 `enhanced` 输出评测值。

表 4: IE 三阶段整体 P/R/F1 对比

阶段	Precision	Recall	F1
Stage 1: baseline	0.3321	0.4348	0.3766
Stage 2: enhanced-regex	0.5971	0.6102	0.6036
Stage 3: enhanced-gliner	0.5860	0.6102	0.5978

这个结果不算“模型加进来就大获全胜”的故事。真正把 F1 从 37.66% 拉到 60.36% 的，是 alias 扩展、词表归一化和字段 fallback。GLiNER 在当前评测集上没有再把总分抬高，反而因为 Precision 略掉了一点，最终 F1 比 regex 低 0.58 个百分点。

5.2 分字段 F1 对比

图 4 把五个自动评测字段的 F1 拆开看。平台字段的提升最猛：baseline 只有 0.4118，到了 regex 阶段直接到 1.0000。GLiNER 阶段四个字段维持不变，只有 output_formats 从 0.4375 掉到 0.4000。

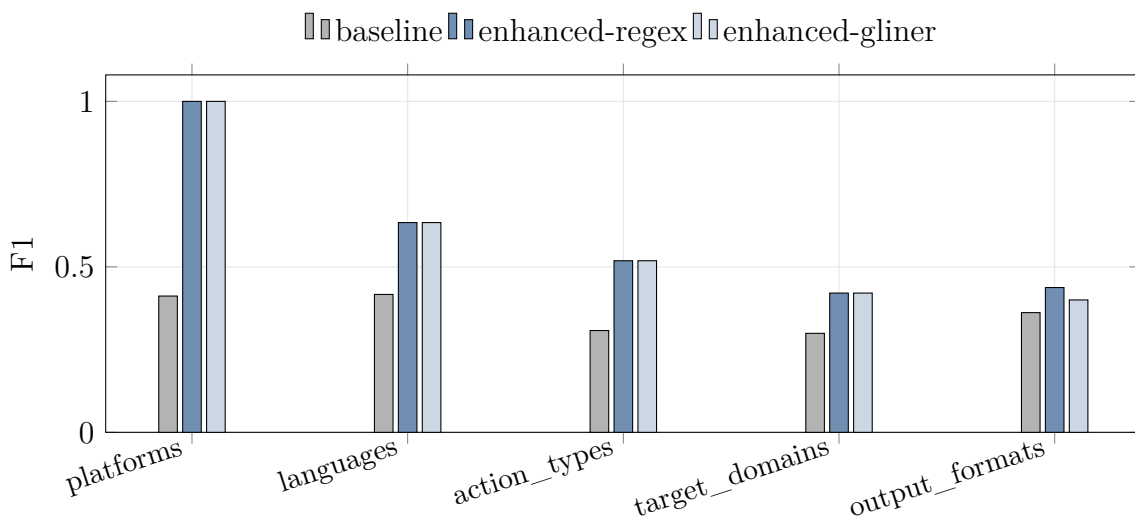


图 4: 五个自动评测字段的 F1 柱状图：规则增强层把平台、动作和语言字段拉高了一大截；GLiNER 阶段在当前标注集上没有继续扩大优势，反而让 output_formats 略有回落。

这张图和表 4 放在一起看，结论就比较清楚了：当前评测集更奖励“可控的规则增强”，而不是“模型先猜一遍再交给规则兜底”。这不代表 GLiNER 没价值，只能说明现在的标注样本还没有把它的价值测出来。

5.3 全量 1000 条覆盖率

为了不只盯着 9 条可对齐样本，我又看了全量 extraction_report.json。当前 enhanced 输出在 1000 条文档上的字段覆盖率是：

- platforms: 24.5%
- languages: 44.0%
- action_types: 87.2%
- target_domains: 92.7%
- output_formats: 47.2%
- metrics: 13.7%

证据来源分布也挺有意思。全局最多的是 `keyword_fallback` (8362 条), 其次是 `gliner_unknown` (5062 条)、`alias_fallback` (1900 条)、`gliner_direct` (1624 条) 和 `gliner_normalized` (582 条)。这说明 GLiNER 其实命中过不少 span, 但很多 span 最后停在了“模型看见了, 词表收不下”这一步。

5.4 误差分析

这部分我只挑最典型的坑, 不把所有 bad case 都平铺出来。

1. 关键字过宽带来的 FP `domain-authority-auditor` 在 baseline 下的 `action_types` 特别典型。标注期望只有 `audit/detect/evaluate`, 系统却能抽出 `build`、`compare`、`generate`、`report`、`review`、`validate` 一大串。问题不在正则写错, 而在这类技能文档本来就像操作手册, 动词太密了。只要看到词就收, Precision 一定扛不住。

2. 词表覆盖不够, FN 还是会留下 `frontend-design` 的 `target_domains` 期望里有 `ui`, 系统长期抽不出来; `manim-composer` 的 `languages` 期望里有 `python`, 系统只抓到 `manim`; `write-xiaohongshu` 的 `target_domains` 期望是 `content/copywriting/social media`, 结果当前输出还是偏到 `ai/frontend`。这些例子都说明词表还不够细, 尤其是“领域词”和“技术词”之间有不少边界模糊的地方。

3. GLiNER 命中了, 但归一化层没有接住 全量输出里一共有 5062 条 `gliner_unknown`, 其中 `platforms` 2310 条、`languages` 1639 条、`output_formats` 585 条、`action_types` 512 条、`target_domains` 16 条。样例里能看到不少很真实的卡点:

- `geo-content-optimizer` 里出现 `ChatGPT`、`Claude`、`Perplexity`, 模型看到了, 但平台词表里没有;
- `schema-markup-generator` 里有 `Schema.org`, 被打成 `platforms` 的 `unknown`;
- `actionbook` 里有 `CDP`、`Unix`, 模型给了 `languages` 候选, 但当前词表不认。

4. GLiNER 也会带来新的误判 `manim-composer` 在当前 `enhanced` 输出里多了一个 `output_formats=markdown`, 而 `regex` 阶段是空值。还有 `frontend-design`, GLiNER 版本把 `output_formats` 从 `html,json` 收缩到只有 `html`, 虽然去掉了一个 FP, 但也没把期望的 `css` 补回来。换句话说, 模型不是纯增益项, 它也会改变规则层本来已经形成的平衡。

6 总结与展望

这次 IE 实验最直接的结论是: 规则层改得对不对, 比模型是不是接进来更关键。baseline 的问题主要是“见词就收”, Precision 被拖得很低; `regex+alias` 一接上, 平台、动作和语言字段马上好看很多。GLiNER 在当前标注集上没有继续抬高总分, 但它留下了不少有价值的候选和 `unknown` 证据, 这些信息对下一轮扩词表很有用。

系统现在的短板也很集中。标注集只有 10 条, 而且其中 1 条缺失在当前数据快照里; GLiNER 只在英文主导文本上启用, 中文和 `mixed` 文本基本还是靠规则; 平台和语言词表偏窄, 遇到 ChatGPT、Claude、Perplexity、Schema.org 这种新词就会掉进 `gliner_unknown`; CPU 上的模型冷启动也比较慢, 不适合频繁大批量重跑。

后面如果继续做, 我会优先补三件事。第一, 扩受控词表和别名, 尤其把这次 `unknown` 里反复出现的实体收进去。第二, 把 `metrics` 单独做一套更像样的评测, 不再只停在“能抽出来几个”这个层面。第三, 如果机器条件允许, 直接换 GPU 跑 GLiNER, 再看看阈值和英文偏置是不是还能放宽一点。

参考文献

[Zaratiana et al.(2023)Zaratiana, Tomeh, Holat, and Charnois] ZARATIANA U, TOMEH N, HOLAT P, et al. Gliner: Generalist model for named entity recognition using bidirectional transformer[A]. 2023.